

Research Report

FactBank: In-Artifact Landmine-Fact Injection for Version-Correct Code Generation

A technical research report and foundation for a future paper

By Mohannad. This report was drafted by an AI assistant, because I do not yet know how to write a formal research paper. The project, its ideas, design decisions, and solutions, are mine; the AI helped with the coding, the brainstorming, and with writing this report. Every figure is traced to a file in the repository, and where a number is hedged (non-Docker harness, unlogged control, single probe) I keep the hedge.

Disclosure

To be transparent about how this document was produced: the FactBank project, its core idea, its design decisions, and the solutions to the problems it ran into, are mine. An AI assistant helped with the coding and with the brainstorming during the project. I do not yet know how to write a formal research paper, so this report was drafted by an AI assistant from the project's own files; it is meant as a faithful, well-organized foundation to build a paper on, not as a finished academic submission. Every number below is traced to a specific file in the repository, and every hedge the project records (for example a non-Docker benchmark harness, an unlogged control, or a single-probe result) is preserved rather than smoothed over.

Abstract

FactBank makes a stock, un-fine-tuned large language model answer correctly about fast-moving software libraries released after its training cutoff. Instead of retraining the model or attaching an external retrieval-augmented-generation (RAG) service, the method distills each library's breaking changes into a small bank of **landmine facts** (post-cutoff changes, habit reversals such as renames, moved imports and flipped defaults, and silent-failure traps), each grounded to a verbatim source quote, and injects the relevant facts at inference time. In the shipped configuration the whole retriever plus the bank is baked into the model's GGUF chat-template, so retrieval runs inside the model with no companion service and no weight change.

The headline results are all hand-verified. On three security and networking expert banks, the baked model's accuracy on domain landmine questions rises substantially at every model size (for example offensive-security 12B 27.3% to 97.7%; eBPF data-plane 26B 42.6% to 91.5%; network-security 2B 39.6% to 66.7%). On the execution-scored GitChameleon 2.0 benchmark, baking a 4,167-fact bank lifts pass@1 by +6.4 points on a 12B model and +2.8 on a 26B model with no weight change, and a two-pass variant reaches 54.2%. A fourth, larger **application-security expert** was then built, baked and shipped at three sizes: a faceted concept-to-variant bank of insecure-by-default landmines that makes the model write secure code without being asked, and, benchmarked on the external SecurityEval suite, the 12B and 26B baked models

beat a much larger cloud model (DeepSeek-V4) on the common pattern-judgeable subset. The project also documents a mechanistic, reproducible finding named the **reasoning paradox**: enabling chain-of-thought makes a habit-reversal fact bank worse, because deliberation re-weights the decision toward the model's trained prior; and it then reports an engineering **cure that makes thinking-on usable** (a two-bug fix that closes the reasoning spiral), shipping both a thinking-off and a thinking-on edition. Where a template outgrew the raw-GGUF metadata limit, delivery moved to a Hub model.yaml route that carries a 4.18 MB template with no cap. Throughout, internally-fair deltas are kept separate from cross-system comparisons, served-loop retrieval ceilings are labelled distinctly from shipped-GGUF numbers, and the threats to validity are listed honestly.

1. Problem and motivation

A model's parametric knowledge is frozen at its training cutoff, but libraries keep shipping breaking releases. Between cutoff and deployment a library may rename an attribute, remove a function, flip a default, or change a return type with no error at the call site. A model that learned the old API emits it confidently, and the failure is quiet: the code parses and the imports resolve, and it breaks only at runtime or, worse, computes the wrong result silently. The project formalises the dangerous cases as three landmine classes: (i) **post-cutoff** (shipped after training, so unlearnable); (ii) **reverses-habit** (a rename, moved import or flipped default the model still knows the old way); and (iii) **silent-failure** (wrong but runs, with no error to self-correct against). A closely related class, elevated to a first-class use case by the application-security expert, is the **insecure-by-default** trap: an API whose default is unsafe (an unpinned deserialiser, an XML parser that resolves external entities, a non-constant-time comparison), where the model's habit and the language's default agree on the wrong answer.

The standard remedies each cost something. Fine-tuning edits weights and must be redone per update; document-RAG externalises knowledge into a live store and returns prose the model must still interpret. The narrower question this project asks is whether the correction can ride **inside the model artifact**, so a stock file, loaded anywhere with no companion server, answers version-correctly. The thesis is that knowledge and reasoning are separable: the weights already reason well even at small sizes, and what small models lack is current knowledge, so handing them the exact missing fact at the moment of need recovers most of the gap.

2. Contributions

- **A fact-extraction method**: an LLM-first structured extractor in which the model supplies meaning and deterministic code verifies every field, gated on a verbatim-quote anchor (FIND, extract, repair, check).
- **A faceted concept-to-variant bank whose facts carry code**: the application-security expert stores the bank as weaknesses (CWE concepts) rather than a library list, with verbatim

bad/good code inside the fact and a mandatory adversarial correctness audit; it is indexed by the coding task so it fires on a benign prompt that never names the vulnerability.

- **An in-artifact delivery mechanism:** a baking procedure that rewrites a GGUF chat-template to carry an inverted-index retriever plus the bank, so retrieval runs in-engine with no external service and with untouched weights; plus a Hub **model.yaml** route (`llm.load.promptTemplate`) that lifts the LM Studio raw-GGUF template cap, proven live on a 4.18 MB template.
- **An evaluation** across four published experts (three security/networking experts at three sizes each, plus a cross-model application-security scorecard on the external SecurityEval suite) and the execution-scored GitChameleon 2.0 benchmark, all hand-verified.
- **The reasoning paradox and its cure:** evidence, by convergence to the no-bank answer, that chain-of-thought reduces a habit-reversal bank's benefit, plus a shipped fix (a two-bug root cause and its template-level repair) that makes a thinking-on edition usable, alongside the thinking-off default.
- **An engineering account** of running a keyword search engine inside a Jinja chat-template: the term-weighting, the gate, determinism, the size wall, and the measured trade-offs.

3. System architecture

The design separates three roles. The **fuzzy core** is the model's weights: reasoning and code structure, never asked to recall an exact post-cutoff fact. The **frozen bank** is a read-only, version-tagged, quote-grounded JSONL fact set. The **code-driven loop** is plain code, not a model tool-choice: it decides to retrieve and places the results in context before the model answers.

3.1 Three retrieval systems: one shipping, one legacy, one test bench

The repository contains three distinct systems that are all loosely called the FactBank bake. They are kept apart deliberately, because the system that produced the benchmark numbers is not the one described by the richer documented recipe.

System / code	Status	Mechanism	What it produced
Baked-Index (<code>bake_index.py</code> , <code>inserts/gemma4_idx</code> <code>/*.jinja</code>)	CURRENT, shipping	Inverted-index retriever inside the GGUF chat-template, run by <code>llama.cpp</code> ; hard library gate; single-query term match; top-5 injected	Every published model and all base to baked benchmark numbers
Static-Bake (<code>package/factbank/bake.py</code>)	LEGACY, superseded	Dumps the whole bank into one static system prompt; no retrieval, gate or ranking	Rejected for serving (roughly 6k-token per request tax, F-040); dormant since the July-15 reorg; it never shipped a model

Served-Loop (extractor/lookup.py)	TEST BENCH, not a bake	A pure-Python retrieval prototype (soft doors, pointer chains, MMR, double-key HyDE) that scores facts for queries and prints them	The offline hand-read retrieval results, the Qwen authority probes, and the application-security retrieval-method ceiling (the served draft-key numbers); it validates facts without baking
---	---------------------------	---	--

So the documented soft-doors, pointer-chains, MMR and double-key (HyDE) recipe belongs to the Python **test bench**. The path that actually ships is a simpler hard-gated, single-query inverted index. This gap is not cosmetic: for the application-security expert it is now named and quantified, because the single biggest lever for benign security prompts (the HyDE draft key) can only run in the served loop, not inside the template. Closing that gap in-engine is open work, tracked as finding F-050.

4. The fact-extraction pipeline

The guiding principle: the LLM does the meaning; deterministic code verifies every field.

4.1 The one law: source targeting

Before anything is extracted, the pipeline targets the library's official **migration or breaking-changes guide**, not release notes and not a blog. This is stated as the one law: source targeting decides everything, because a perfect extractor on the wrong document yields a bank of the wrong facts. A library with no prose upgrade doc is treated as un-mineable, which is why kymatio, lightgbm and tqdm stay uncovered. The application-security expert sharpened the law: source targeting also applies **inside** a source, curating within a large standard (CWE) and gating a source's low-bite entries (RustSec's memory-safety CVEs), because a permissive corpus is not uniformly worth mining.

4.2 Extract, repair, check

Extract is the only LLM stage, one call per chunk. The backend is DeepSeek deepseek-v4-flash with thinking off, driven by function-calling (emit_facts) rather than strict JSON-schema, which DeepSeek does not support; temperature 0.2; the source is chunked at 3,500 characters split on blank lines; max_tokens 8,000. The extractor is OpenAI-compatible: LLM_BASE_URL, LLM_MODEL and LLM_API_KEY point it at any compatible endpoint (DeepSeek is the default), and the DeepSeek-only thinking:{disabled} field is sent only to DeepSeek. Extraction costs about 4 cents per library.

Repair is a checker with no LLM. When the model paraphrased a quote, the pipeline re-grounds it by snapping the quote to the nearest real source line, and admits that snap only if the line contains one of the fact's own symbols and shares at least two content tokens. Check has no LLM either: it validates fields and enums and enforces the anti-hallucination anchor, so the

canonicalised quote must be at least 12 characters and a substring of the canonicalised source, otherwise the fact is dropped to a .rejects file.

4.3 Schema: what the model fills vs what the code derives

Field	Filled by	Notes						
subject, old, new, truth, why_it_bites, quote, associative	the LLM (function call)	meaning only; why_it_bites in {post-cutoff, reverse-s-habit, silent-failure}						
type	code	reconciled from presence of new: REMOVED, REPLACED, or CHANGED						
keywords.from_fact	code	symbols pulled from old, new and truth (always grounded)						
id	code	sha256(lib)	version	type	subject	old	new	truth[:80][:10]; global uniqueness is a CI gate
lib, version	code	stamped from CLI args						

Deriving type and from_fact in code rather than trusting the model's enum is the design-over-output-mode decision, and it is load-bearing: on a Kubernetes source it moved cleanliness from **35% to 96.5%** with the same model and prompt.

4.4 Extractor 2.0: code lives in the fact

The application-security expert introduced a second-generation extractor whose defining move is that the **code lives inside the fact**. Instead of a separate code search at serve time, a fact optionally carries its own verbatim code_bad / code_good pair, grounded to the source modulo whitespace and never canonicalised. One shared pipeline (a single core run()) drives thin per-source adapters, retiring the earlier parallel-agent mining; grounding is done at the sentence boundary rather than the ellipsis-only pass of the v1 repair; and cross-source dedupe is prose-only, so two facts sharing a code_bad are not treated as duplicates. The one anti-hallucination law is unchanged (a verbatim quote anchor), but a new, mandatory step was added on top of it, described in 4.6.

4.5 Measured extraction quality

Source or effect	Result
k8s deprecation guide (clean prose)	83/86 = 96.5% clean (the older rules-miner got 44)
pydantic v1 to v2 (messy prose, code, tables)	145/178 = 81.5%, then 174/178 = 97.8% after repair
transformers release notes	130 facts (rules-miner got 8)
GitChameleon expert, 3 checker bugs fixed	recovered +901 facts (+28%): a DEPRECATED-enum leak; a canon markup bug (777 false not-verbatim rejects on rst/sphinx); repair anchors too narrow
the count lies (pydantic)	a raw 58% hid about 51 good facts behind 2 checker bugs, found only by reading rejects
reachability (model-free)	69.6% with a door hint; door purity 94.5%

4.6 Grounding proves real, not correct: the adversarial audit

The quote anchor proves a quote is real, not that it is still true. A migration guide is historical, and a documented change can be reverted. This came up in practice on 2026-07-17: an ldap3 fact saying the default Connection strategy changed from SYNC to RESTARTABLE described a reverted change (the current default is SYNC), and it was removed before it could mislead. A stale fact corrects the model to the wrong answer, which is worse than no fact. Extractor 2.0 answered this systematically: after grounding, every fact is put through a **mandatory adversarial correctness audit** (a Sonnet pass, 41 verifiers plus a per-flag re-check) whose explicit premise is that grounding proves REAL, not CORRECT. On the application-security bank it removed about **3.8%** of facts as wrong or stale. This is the operational form of the project's currency rule: verify a habit-reversal default against the library's current source, not just against the historical guide it was quoted from.

4.7 What worked and what was rejected

Kept: schema-design over output-mode; function-calling; the verbatim-quote anchor; the repair snap (now sentence-boundary in 2.0); two-tier keywords; code-in-the-fact; migration-guides over release-notes; rich, non-model-gated banks; the adversarial correctness audit; and, in the Served-Loop test bench, soft doors plus pointer chains plus richer keywords plus MMR plus double-key HyDE. Rejected, each with a measured reason: hard doors (a wrong top-1 amplifies to an all-wrong result, so the design fails open instead); rare-token-squared weighting (it did not fix targeting); the concrete-rewrite second pass (only about 5% of facts are vague, all from release notes, which is a source property); the landmine-gate that drops what the model already knows (it over-fits one model, since a 3B lacks a 12B's knowledge); and hand-built alias tables (redundant once keyword extraction is rich).

5. The bank: storage, provenance, policy

The master bank is plain JSONL, one fact per line. Facts carry two provenances and the distinction drives everything. In the shipped Python-data model the bank is 1,911 facts = 388 curated + 1,523 mined.

Provenance	Origin	Evidential worth
curated	hand or agent extracted from real docs; verbatim-quote gated	high: renames, removals, gotchas; the facts that beat a wrong prior
mined	inspect.signature on the installed library plus its Sphinx objects.inv URL	unproven: mined facts have won zero eval cases (F-065); hallucination-proof but shallow

The mined pool at large is about 66% junk (F-064): the miner walks the inheritance chain and donates inherited methods to the wrong class (str.join entered as a matplotlib fact), which flattens IDF and crowds the top-k; on the data domain 24,133 mined facts were gated down to 8,096. The project's position is that a bigger mined pool is not an upgrade. The policy is **landmine-only, no fundamentals**. Because every fact targets a post-cutoff API, any correct answer must have come from the bank, which is what makes the measurements mean something.

The value function for choosing which experts to build: a bank pays off where the model is most likely to be wrong, which is fast release cadence times habit reversals times silent failures times a mineable migration guide, and, as a later study sharpened it, times **low training representation** (niche or elite tooling the internet does not document for free). This is why the netsec wins concentrate on genuine rewrites such as volatility3 v2 to v3 and yara-x, and why the dataplane edge model starts at just 10.6%.

5.1 The faceted concept-to-variant schema (application-security)

The application-security expert stores the bank differently, and the difference is the point. Instead of a flat list keyed by library, the bank is **faceted by weakness**: a concept (a CWE) fans out into variants, so the shipped bank `FINAL_v3.jsonl` is **258 concepts (254 CWE plus 4 synthetic door:* groups) that expand to 3,984 variants**, of which **1,075 carry verbatim bad/good code**. Organising by weakness rather than by a short library list lets one bank span **10+ languages** (Python 1,315, Java 729, JavaScript 360, C 290, Swift 230, Go 191, C# 139, Rust 98, Ruby 97, and more) and many frameworks (Android, iOS / SwiftUI / WKWebView, Flask, Django, Express, Node, ASP.NET / .NET, Spring, Java EE, Rails, Laravel). The facts are insecure-by-default landmines, so the model writes secure code without being asked: `torch.load(weights_only=True)` (a post-cutoff default), `XXE resolve_entities=False / no_network`, `yaml.safe_load` over `yaml.load`, secrets over random, parameterised SQL, `os.environ` credentials over hardcoded, `ast.literal_eval` over `eval`, constant-time HMAC. The **benign-prompt bridge** makes this work: because each fact is indexed by the coding task (feature phrases and use cases), the bank fires on a benign prompt that never names the vulnerability.

Provenance for this bank is seven permissive sources: MITRE **CWE**, GitHub **CodeQL**, SAST (**Bandit** and **gosec**), OWASP **MASTG**, **RustSec**, **NIST / RFC / Mozilla** crypto-net guidance, and **OWASP**. Each fact is quote-grounded to its source and then survives the adversarial correctness audit of 4.6.

6. Baking: the in-template inverted index

6.1 Term weighting at build time

Each fact becomes weighted index terms; the weight expresses how strongly a word proves this is the wanted fact. A fact is treated as dead when its kind is mistake or its text contains renamed, removed, deprecated, no longer, or replaced.

Term source	Weight
dead API name (rename or removal, rare, df at most 3)	10
rare API token (df at most COMMON_DF = 8)	4
common token	1
description word, curated fact vs mined fact	3 vs 1
task-phrase word vs bigram	2 vs 3
Doc2Token expansion word (length at least 5) vs bigram	1 vs 2

Four transformations follow, each present because its absence broke something. (1) **Squash-normalisation and identifier splitting**: `ttest_ind` also indexes as `ttest`, `ind` and `ttestind`, and `scikit-learn` as `scikitlearn`; only weight-at-least-4 tokens earn variants, because squashing everything blew the index from 11k to 18k terms. (2) **Doc2Token expansions**: the model wrote everyday questions for its own curated facts offline, and only bigrams and length-at-least-5 unigrams are

indexed below real terms, because dumping all words blew the template to 1,077 KB. (3) **IDF at build time**: a term at 3 or fewer postings gets times 3, at 10 or fewer times 2, else times 1, and above 40 postings it is dropped (DF_CAP), so the runtime stays integer addition. (4) **Control culling (F-046)**: any word that appears in the control questions is deleted from both index and gate. The build is deterministic (sorted iteration plus sort_keys), and a sanitize step strips repr() memory addresses that had made 13 facts non-reproducible.

6.2 Index statistics as shipped

Bank	Index and template
Python-data (1,911 facts)	17,260 terms, 34,671 postings (2.0 per term); about 540 KB index; 973,652-byte template (388 curated, 1,523 mined)
GitChameleon (4,167 facts, 23 doors)	58,877 terms; 2.73 MB template
netsec (114 facts, 7 libs)	about 2,317 terms; about 118 KB template
application-security (3,984 variants, faceted)	4.18 MB template (over the raw-GGUF cap; ships via model.yaml)

6.3 The gate and its aliases

The gate opens phone-book tabs: a library named in the query opens only that tab (named always wins); otherwise tabs open on trigger words. Each library carries its name plus up to 48 unique, control-safe trigger words (curated first) plus aliases. A real fragility surfaced and was fixed here: the gate opens on the exact token, so a natural or old name (BloodHound, Volatility 3, CrackMapExec) opened no tab and injected **zero facts**. The fix (gen_gate_aliases.py feeding bake_index.py --extra-aliases) makes every rename's OLD name a gate alias (netsec 100 aliases, offsec 199, dataplane 178). It was verified offline with render_retrieval.py: Volatility 3 injects 0 facts before the fix and 5 after. A narrow rescue map (weight-10 dead API names, one library, control-safe; 657 such terms) fires only when no tab opens. One caveat is recorded honestly: the runtime canonical-alias file aliases.json is empty, so the working aliases come from the manual GATE_ALIASES table and from --extra-aliases. The faceted application-security bank sidesteps the library gate entirely: because it is keyed by weakness and coding task rather than by library, the library door is effectively removed and retrieval routes on the concept, which is what lets a benign prompt reach the fact.

6.4 The size wall

There is exactly one residual wall, and it is one application's bug, not a format or engine limit. LM Studio's raw-GGUF metadata reader silently swaps a chat-template over **980,000 bytes (957 KiB)** for a 48-character sentinel, so the model loads fine and answers garbage (F-053). llama.cpp caps at 1 GiB and errors loudly; the LM Studio Hub model.yaml route (llm.load.promptTemplate) has no cap, proven live at 1.5 MB and 2.0 MB (F-059) and now at the application-security expert's **4.18 MB** template, the first shipped expert to require it. The baker enforces the cap only as an opt-in hard-fail on --route rawgguf; the llamacpp and hub routes have no cap. A roughly 100-fact bank is about 115 KiB and works everywhere; the 2.73 MB GitChameleon and 4.18 MB application-security banks need the model.yaml route for LM Studio only.

7. Runtime retrieval and delivery (Baked-Index)

llama.cpp's Jinja engine runs the following on the last user message, before the question renders:

- **Normalise:** lowercase; punctuation becomes spaces (hyphens and underscores are handled by squash).
- **Gate:** named library wins; otherwise trigger-word tabs; no tab means nothing renders, so the request is byte-identical to a stock model.
- **Intent:** set a breakage-intent flag if any query word is in the INTENT set.
- **Build query terms:** content words plus squashed and split variants plus alias canonicals plus adjacent bigrams.
- **Postings lookup:** for each term, add its postings; discard hits whose library is not in an open tab.
- **Score:** integer adds; a curated fact gets +6, or +12 when breakage intent is present (id below FB_NCUR).
- **Top-5** (FB_MAX) by max-selection, then deliver.

The shipped baked path does **prompt-only** retrieval: it searches on the user's question but not on the model's own draft. The double-key HyDE variant (searching additionally on a draft answer, which names the insecure API a benign prompt omits) lives only in the served test bench, because a draft does not exist yet at template-render time. For the application-security expert this is the single largest source of the served-vs-shipped gap, and it is reported as such.

7.1 Delivery lane: a forged native tool exchange

Facts are not delivered as prose. The template renders a completed factbank_search call and tool-response in the model's own tool syntax, before the user's question. The reason is measured: handed the same facts as prose, one glyph-heavy question burned **8,189 reasoning tokens and produced no answer**; through the tool channel the same question took 390 to 757 tokens and answered correctly, because a tool_response is not something the model argues with. Placing the block before the question also fixes an echo bug (F-042) and buys prompt-prefix caching. The injected note also carries the authority frame (marking the facts authoritative and mandatory, to be obeyed over a conflicting trained habit); the wording of that frame turned out to matter for thinking-on, as 8.2 describes.

8. Serving and the thinking-on cure

Serving uses llama.cpp llama-server with --jinja, not LM Studio raw-load. Sampling uses the model's native settings (for Gemma temperature 1.0, top_k 64, top_p 0.95, min_p 0.01), where the min_p floor prevents a reasoning model's repetition-loop empty answer. max_tokens is one shared budget for reasoning and then the answer: if it is exceeded, the backend returns an empty content with finish_reason length and no error, so it is set large. Three Jinja engines exist in this world: Jinja2 (Python, offline dev and test), minja (old llama.cpp, now removed by PR

#18462), and llama.cpp's current C++ Jinja engine (what serves today), so where the notes say minja, read the latter (F-054). jinja2 is not the shipped engine (F-050): the same template once pulled 5 facts under jinja2 and 1 in production, which is why a 22/22 parity check is run.

8.1 The reasoning paradox, restated

The paradox (10.3) is that turning chain-of-thought on makes a habit-reversal bank worse, by two mechanisms: the reasoning spirals and truncates to an empty answer, and where it commits it reverts to the trained prior. For a long time the project's only remedy was to ship thinking-off, or to lean on authority framing. That left a real want: a thinking-on edition that shows its work without collapsing.

8.2 Making thinking-on usable: a two-bug fix

The application-security work isolated the cause as **two stacked bugs**, and fixed both. (a) A known llama.cpp Gemma-4 chat-template bug in the generation-prompt path: with thinking off the template emitted a closed, empty thought block, but with thinking on it emitted **nothing**, so the model was handed no open thought channel and spiralled until the token cap (the empty and truncated answers). The fix is to open the channel in the prompt when thinking is on (emit the thought-channel opener rather than nothing). (b) A weak authority frame in the injected note let the model reason back to its prior and revert the fact; the fix restores the strong frame proven in the served test, marking the facts authoritative and mandatory and instructing the model to override its training defaults and any conflicting habit. A third, delivery-level piece is required: force `enable_thinking` as the template default rather than leaving it to a client flag, because **LM Studio drops chat_template_kwargs**, so the thinking mode has to be baked into the template.

Measured, on the 12B with the same bank: empty answers fell from **5/30 to 3/30** and truncated answers from **12/30 to 4/30**, with responses now closing cleanly at roughly 2 to 2.8k reasoning tokens; an e2b thinking-on smoke test was 8/8 clean. A residual remains and is stated plainly: about **10% of the hardest-reasoning prompts still spiral to a blank answer** even at a 12k-token budget. That is inherent to reasoning-on, and it **fails safe** (the output is a blank, never insecure code). Note the failure mode is distinct from the low-temperature repetition loop that `min_p 0.01` cures; `min_p` does not fix the reasoning spiral.

8.3 Ship both editions

The owner's decision is to ship **both** editions, differing only by the `enable_thinking` default. Thinking-off is the fast, reliable default; thinking-on adds a visible reasoning trace at the cost of that ~10% blank-but-safe residual. Both are shipped for the application-security expert at all three sizes.

9. Experimental methodology

Multiple benchmarks are used. First, three security experts, each with a set of landmine questions (netsec 48 = 30 easy + 18 hard; offsec 44; dataplane 47), phrased so the model's old

trained answer is wrong. Second, **GitChameleon 2.0**, which is 328 version-pinned Python problems over 26 libraries, execution-scored by hidden pytest (arXiv 2507.12367). Third, for the application-security expert, the external **SecurityEval** suite (s2e-lab, MSR 2022, 121 Python CWE tasks), scored on the common pattern-judgeable subset against a cloud baseline.

Scoring is by hand. An automated substring or pattern scorer is used only to triage what to read; the verdict is a person reading the model outputs. The scorer errs in both directions: it false-fails a correct answer that names the old API in an explanatory comment or as a contrast, and it false-passes a semantically wrong answer (an offsec case where the model inverted the fact). Standing rules: an old name in a migration comment is a pass; an empty answer is a fail; borderline calls go to the base model's favour, which makes the reported lift a lower bound.

Base vs baked is clean because the baked template injects facts internally, so the prompt is byte-identical for base and expert, and each size is compared against its own base. The security-landmine runs use thinking-on plus an authority system prompt, Gemma-native sampling, and 6-way parallelism. The GitChameleon base and baked headline runs are thinking-off greedy. The cross-model SecurityEval runs are thinking-off, and the DeepSeek-V4 baseline is run in no-thinking mode to match.

10. Results

10.1 Security experts: the 3-size curve (hand-scored)

Expert	2B (e2b)	12B	26B (A4B MoE)
netsec /48	19 to 32 (39.6 to 66.7%), EC 45%	27 to 39 (56.2 to 81.2%), EC 57%	37 to 45 (77.1 to 93.8%), EC 73%
offsec /44	12 to 37 (27.3 to 84.1%), EC 78%	12 to 43 (27.3 to 97.7%), EC 97%	17 to 43 (38.6 to 97.7%), EC 96%
dataplane /47	5 to 32 (10.6 to 68.1%), EC 64%	16 to 41 (34.0 to 87.2%), EC 81%	20 to 43 (42.6 to 91.5%), EC 85%

EC is error-closure, the share of the base's wrong answers the bank fixed. Two consistent trends appear: the raw lift shrinks as the base grows, but error-closure rises (netsec 45 to 57 to 73%), so small models need the bank most and big models use it best. The downsides are reported too: about 3 of 47 dataplane-12B cases regressed (the base was already right and the injected fact plus reasoning made it worse, the reasoning-paradox tax), and netsec had 1 regression each at e2b and 12b and 0 at 26B.

Auto-triage vs hand-score (why every answer is read)

Run (baked)	auto	hand
netsec 12B	35/48	39/48
offsec 12B	37/44	43/44
offsec 26B	39/44	43/44
dataplane 12B	35/47	41/47
dataplane 26B	39/47	43/47

Reading by hand moved the baked scores up by 3 to 6 per transcript, correcting false-fails (the old API named only in a comment) and catching at least one false-pass (a semantically inverted answer the substring scorer accepted).

10.2 Application-security expert: cross-model SecurityEval (shipped GGUFs, thinking-off)

This is the first FactBank expert benchmarked against an external suite and a cloud baseline. SecurityEval has 121 Python CWE tasks; the common pattern-judgeable subset where all arms are machine-scoreable is **21 tasks**, and the cloud baseline DeepSeek-V4 (deepseek-v4-flash) is run in no-thinking mode to match. Patterns triage; every win is hand-verified.

Model (baked, thinking-off)	secure / 21
e2b base	11
e2b + bank	13
12b base	17
12b + bank	17
26b base	17
26b + bank	19 (best)
DeepSeek-V4 (cloud, no-thinking)	14

Head-to-head against the cloud model, the bank is **26b +5 / -0, 12b +3 / -0, e2b +2 / -3**: so 12B+bank and 26B+bank beat the much larger cloud model, and e2b+bank roughly ties it. Per-size lift on each model's own judgeable set is **e2b 15 to 18, 12b 29 to 31, 26b 30 to 33** (+2 to +3), with an XXE (CWE-611) sweep on the bigger models (12b 5 to 6/6, 26b 4 to 5/5). The honest reading: a bigger model is more secure to start with, and the bank helps most where the base is weakest and on specific weaknesses (XXE, deserialization, weak-random, hardcoded creds). The bank fires (is active) on 116 of 121 tasks; the modest e2b lift is a genuine 2B-obedience limit, not a retrieval failure, and that was verified by hand.

Served ceiling vs shipped artifact (a methodological-honesty note)

The shipped numbers above are prompt-only, in-template retrieval. A separate, **higher** set of numbers exists for the **served-loop** prototype, and they must not be read as the shipped GGUF. The served loop adds the HyDE double key (retrieve on the prompt AND the model's own draft, which names the insecure API a benign prompt omits) plus authority framing plus thinking-on, and it reaches a higher ceiling: base 14 to **25** secure on a 28-task SecurityEval pattern-subset, and e2b served-bank **22** vs DeepSeek-V4 13 on a 24-task subset. These are the retrieval-**method ceiling**, not the shipped artifact; the shipped baked GGUF cannot run the draft key inside the template, so it scores lower (the 21-task table above). The gap between the two is exactly the value of the draft key, and it is the concrete, quantified form of the recipe-vs-shipped gap from 3.1.

10.3 GitChameleon 2.0: execution pass@1

Model (Q4, llama-server)	base	+ baked (thinking-off)	delta
--------------------------	------	------------------------	-------

gemma-4-12B (249 buildable)	94/249 = 37.8%	110/249 = 44.2%	+6.4 (fixed 30, broke 14)
gemma-4-26B-A4B (249 buildable)	108/249 = 43.4%	115/249 = 46.2%	+2.8 (fixed 21, broke 14)
12B, pessimistic all-328 basis	28.7%	33.5%	+4.8

The bank here is 4,167 facts across 23 doors, 100% quote-grounded, in a 2.73 MB template. There were zero empty answers on either side of the thinking-off runs, so no artifact inflates the gap. To be clear, the +6.4 is a mix of direct fact-application and prompt-perturbation variance.

10.4 The reasoning paradox

Baked 12B configuration	pass@1 (249 buildable)
thinking OFF (shipped)	110/249 = 44.2%
thinking ON (temp 0.6)	92/249 = 36.9% (down 7.3)
two-pass: authority plus thinking on the misses	135/249 = 54.2% (up 10.0 vs off; up 16.4 vs base)

Two mechanisms account for the loss. First, reasoning spirals and truncates: 99 of 328 (30.2%) thinking-on answers came back empty (finish_reason length); short-reasoning problems (under 4k characters) pass 73%, long ones (over 16k) pass 14% (56% empty), and raising the budget only lengthened the spiral (14k to 28k characters). Second, where it commits, reasoning reverts to the trained habit, shown by **convergence**: the baked-plus-thinking answer equals the no-bank base answer (falcon status.body vs .text; pandas groupby observed). On the 185 problems where thinking-on committed an answer, its accuracy is a statistical tie with thinking-off (49.2% vs 50.3%), so reasoning adds no accuracy (the knowledge is in the injected fact), only variance and the reversion mode. The two-pass 54.2% uses a ground-truth-failure oracle to choose which to re-run (unrealistic in production) and costs 2x compute on the recovered slice. The engineering cure in 8.2 does not overturn this paradox on GitChameleon; it makes a thinking-on edition survivable (far fewer blanks and truncations) so that, where a task genuinely needs a reasoning trace, it can be shipped as a labelled alternative with a known ~10% blank-but-safe residual.

10.5 Authority framing and a 4B thinking model

On a thinking-native model that cannot disable reasoning (Qwen3-4B-Thinking-2507, Q8_0), the falcon .body to .text landmine was probed under four conditions (control, authority-in-prompt, authority-via-tool, both), 3 reps each on 2 problems. All three **framed** channels pass: the retained log is exactly 18/18 framed rows, and the 0/6 unframed control was read from a session but not saved as a log, so it is treated as illustrative, not measured (the project warns against citing a 30/30). On a four-problem difficulty ladder: easy (torch boolean mask) passes; medium (networkx greedy modularity) passes via a rescued fact; hard (torch.stft return_complex, solved by only 1 of 4 gemma configs) passes with the fact cited; impossible (torch 1.9.0 log_ndtr, numerical stability) fails, which is outside what any fact bank can supply. Full 328: Qwen-4B plus bank plus authority scores 31.3% (78/249 buildable) with zero empties, flagged as flattering (Q8_0 vs the gemmas' Q4_0) with no base run, so the lift is unmeasured.

10.6 Frontier context (not a ranking)

Official GitChameleon leaderboard (328, Docker, hidden tests)	greedy	+RAG
o1	51.2%	n/a
Gemini 2.5 Pro	50.0%	56.7%
GPT-4o	49.1%	n/a
Claude 3.7 Sonnet	48.8%	56.1%
GPT-4.1	48.5%	58.5%
Claude 4 Sonnet	n/a	59.4% (best)

Comparability caveat: the FactBank rows come from a local, non-Docker harness over the 249 problems that built (some 3.7 remapped to 3.9, no RAG, hand-scored), while the frontier column is a different measurement (all 328, Docker, official scorer). Base vs baked is internally fair; the frontier column is read as which neighbourhood the approach is in, not as a ranking or a claim of beating those models. For orientation, 12B plus bank at 33.5% over all 328 lands near Llama-3.3-70B territory (36.3%), a model about 6x larger.

10.7 Engineering results (serving)

Measurement	Result
Inverted index vs the linear scanner (same bank)	gold 9/12 vs 6/12; 809 KB vs 949 KB; 545 ms vs 2,629 ms per matched request
Facts in a list vs a dict	291 ms vs 1,318 ms per request
Prompt-prefix caching (repeat topic)	TTFT about 1.8 s down to 0.24 s
Shipped 1,911-fact model gates	gold 12/12; controls 0/10 false fires; lint 8/8; jinja2 to engine parity 22/22; TTFT 0.18 to 1.91 s
Scaling and the real wall	21k facts render a matched question in 218 ms (latency is not the wall); gold recall decays 12/12 to 9/12 as the bank grows to about 21k (precision-at-density is the wall, F-056)
Thinking-on enablement (12B, appsec bank)	empty 5/30 to 3/30; truncated 12/30 to 4/30; clean closes at ~2 to 2.8k reasoning tokens; ~10% residual blank-but-safe

11. Built banks, publishing, and provenance

Ten-plus banks have been built (about 90 libraries and a large weakness-faceted corpus). **Four experts** are now baked, hand-scored and published: the three security/networking experts and the application-security expert.

Expert	libs / scope	facts	status
AI-ML	11	817	built; not baked
GitChameleon	23	4,167	built, baked, benchmarked

Web (full-stack)	9	279	built, checked; not baked
netsec (security and networking)	7	114	baked x3, tested, PUBLISHED
offsec (offensive security and RE)	17	489	baked x3, tested, PUBLISHED
dataplane (eBPF and data plane)	7	318	baked x3, tested, PUBLISHED
application-security (appsec)	258 concepts (254 CWE), 10+ languages	3,984 variants (1,075 with code)	baked x3, BOTH editions, benchmarked, PUBLISHED
DevOps / Databases / Web3	3 / 1 / 2	74 / 32 / 48	built, checked

The three security experts are published as 9 Hugging Face repos, gemma-4-{E2B,12B,26B-A4B}-{netsec,offsec,dataplane}-expert-GGUF, in one collection; license gemma (these are gemma-4 derivatives, weights untouched, only the chat-template carries the bank); provenance stamped factbank.version 0.4.0; each netsec repo keeps its pre-gate-fix bake as tag v1-pre-gate-fix.

The application-security expert is published as **6 LM Studio Hub virtual models** gemma-4-{e2b,12b,26b-a4b}-security-expert (each with a -thinking companion), with sampling and context baked into the Hub settings (config.operation.fields: temp 1.0 / topK 64 / topP 0.95 / minP 0.01; config.load.contextLength 32768), and as **3 Hugging Face GGUF repos** gemma-4-{E2B,12B,26B-A4B}-security-expert-GGUF, each holding BOTH GGUFs plus both templates plus both model.yaml files plus a card documenting both editions, in the **Information Security EXPERTS** collection. Because the 4.18 MB template exceeds the LM Studio raw-GGUF cap, this is the first expert to ship over the Hub model.yaml route (6.4, F-059).

The local baked GGUFs were deleted after publishing to free about 89 GB (byte-verified first; Hugging Face is canonical, and any expert re-bakes from base plus bank). GitHub hosts an orphan-snapshot mirror of the v2/ tree only (no archive, no weights).

12. The evidence trail (F-001 to F-065)

The project keeps a numbered chronological log of empirical findings (archive/docs/FINDINGS.md), which are measured results, invalidated measurements, and discovered capabilities, and it is the proof base behind every claim. Findings cited above include F-040 (the whole-bank static prompt was rejected for serving, roughly 6k-token tax), F-046 (control culling), F-050 (jinja2 is not the shipped engine), F-051 (the inverted index beats a linear scan), F-053 (the LM Studio silent sentinel), F-056 (precision-at-density is the wall, gold 12/12 to 9/12), F-059 (the size wall is beaten via model.yaml, now proven at 4.18 MB), F-063 (live obedience 8/8 vs 3/8 bare), F-064 (66% of the mined pool is junk), and F-065 (signature-mined facts win zero cases, so automated checks triage and reading is the verdict). The application-security session added its own records: the faceted concept-to-variant build, the adversarial

correctness audit (~3.8% removed), the thinking-on enablement fix (TICKET-thinking-on-enablement.md), the cross-model SecurityEval scorecard (SCORECARD-crossmodel.md), and the publish artifacts.

13. Threats to validity and limitations

- **Benchmark fidelity:** the GitChameleon numbers come from a local, non-Docker harness (Python 3.7 and 3.8 remapped to 3.9; the venv cache key omits dependencies; 249 of 328 are buildable). The current run_tests.py docstring claims exact-Python and deps-in-key, but the reported figures predate that remediation; base vs baked deltas are internally fair, absolute numbers are local-harness provisional. The 79-unbuildable cause is not isolated (53 of 79 are 3.7 forced to 3.9).
- **Served ceiling vs shipped artifact:** the strongest application-security retrieval numbers (base 14 to 25; e2b served-bank 22 vs cloud 13) are the served-loop **method ceiling** with the HyDE draft key, not the shipped GGUF, which is prompt-only and scores lower (the 21-task cross-model table). The two are labelled distinctly throughout and must stay that way.
- **Recipe vs shipped system:** the best-documented retrieval recipe (soft doors, pointers, MMR, HyDE) is the Python test bench; the system benchmarked is the simpler hard-gated baked index. In-engine parity of the full recipe is open (F-050), and the gap is now named and quantified for appsec (the missing draft key).
- **Value concentrates in curation:** mined facts win zero cases, and the mined pool is about 66% junk. The durable payload is the hand-curated (or audited-extracted) subset, which is small and labour-intensive, so it bounds how far the approach scales.
- **Grounding is not correctness:** a verbatim quote proves a quote is real, not current; the mandatory adversarial audit removed ~3.8% of appsec facts as wrong or stale, and a reverted change baked as a fact corrects the model to the wrong answer (the ldap3 case). This risk is mitigated only by that audit plus a per-fact currency check.
- **Gate fragility:** exact-token gating silently injects nothing on natural or old names; the alias fix is a vocabulary patch that will recur for unanticipated phrasings, and the rebaked security models were not re-scored on GPU (recovery was verified offline only). The faceted appsec bank sidesteps the library gate but leans instead on the coding-task index, whose recall on a large mixed bank is itself under investigation.
- **The reasoning claims are under-powered:** the authority result rests on one landmine (18/18 framed) with an unlogged control and a 4-problem ladder, so the strong general claim needs a broader, logged evaluation. The thinking-on numbers are non-deterministic (temp above 0), and the thinking-on enablement fix still leaves a ~10% blank residual on the hardest prompts (fail-safe, but a real cost).
- **Over-deference, not yet quantified:** every authority probe used a fact that applied, and strong wording like the fact is absolute could force an inapplicable near-miss fact into the answer. The thinking-on edition's stronger authority frame makes this more relevant, not less. This is named as the most important next experiment and has not been run.

- **Cross-model caveats and doc drift:** the Qwen 31.3% is flattering (Q8_0 vs Q4_0) with no base run; there are also minor inconsistencies (associative-count 1 to 8 vs 3 to 8 vs a 5 to 8 prompt; a 115 vs 114 netsec typo; door and lib counts 23 vs 24 vs 26).
- **Application-security bank reproducibility, and a designed-but-unshipped schema:** the v3 build pipeline was reconstructed and committed (2026-07-19) — `build_v3_assemble2.py` plus its now-tracked inputs (`FINAL.jsonl` + the `v3_llm/v3_tier1/v3_usecases` enrichments) rebuild **3630/3984 (91%) of `FINAL_v3.jsonl` exactly**. The ~9% that does not reproduce is irreducibly lost source data (feature_phrases from overwritten `--sample` use-case runs, plus five manual CWE-367 overrides), so the committed `FINAL_v3.jsonl` remains canonical as the exact artifact — an equivalent (not byte-identical) bank is rebuildable from the base facts. Relatedly, the concept-record schema documented in `SCHEMA-V3.md` (a shared lesson plus `sub_patterns`) never shipped: the 258 concept records are `{kind, concept_id, n}` stubs that neither retriever reads (both key off each variant's `concept_id`), so they are inert routing metadata rather than the designed lesson store.

14. Novelty and related work

The work sits between RAG, knowledge editing, and prompt engineering, and the novelty is in the combination. Against **document-RAG**, the payload is curated one-line landmine facts (not documents), retrieval is deterministic (no tool-choice), and it can be baked into the model file with no runtime store. Against **fine-tuning and knowledge-editing**, no weights change (the same base serves with or without the bank, and banks are swappable), but the knowledge lives in context, which is exactly what exposes it to the reasoning-reversion effect. The double-key (HyDE-style) retrieval exists only in the Python test bench, and the application-security expert shows it is the decisive lever for benign security prompts. The most defensible novel claims are (1) in-artifact version correction via a working in-engine inverted index in the chat-template; (2) the reasoning-paradox finding with convergence evidence, the authority-framing remedy, and now a shipped two-bug engineering cure that makes a thinking-on edition usable; and (3) a faceted concept-to-variant, code-carrying bank of insecure-by-default landmines that makes a stock model write secure code without being asked and, externally benchmarked, beats a much larger cloud model at 12B and 26B. The retrieval machinery itself (inverted index, build-time IDF, document expansion, identifier splitting) is competent engineering from known ideas rather than an algorithmic novelty, and the project says so.

15. Reproducibility and artifacts

Key code paths (under v2/): `extractor/{extract,repair,check,run,lookup}.py` and `appsec_core.py` plus the per-source adapters (fact-making, code-in-fact, and the served test bench); the adversarial audit harness; `bake/template-brain-v3.1/bake_index.py` and `inserts/gemma4_idx/*.jinja` (the baked retriever); `bake/template-brain-v3.1/{enrich,designs,gen_gate_aliases,render_retrieval}.py` (term logic, gate aliases, offline render prover); `eval/gitchameleon/run_tests.py` and

extractor/experts/*/test_*.py (execution and landmine harnesses); the cross-model SecurityEval scorecard (SCORECARD-crossmodel.md) and the thinking-on enablement ticket (TICKET-thinking-on-enablement.md); serving/*.md and decisions/*.md (the design record). Determinism (sorted iteration plus sort_keys) makes the **template bake** byte-reproducible **from a frozen bank**. The application-security **bank build was reconstructed and committed (2026-07-19)**: build_v3_assemble2.py plus its now-tracked inputs (FINAL.jsonl + the v3_llm/v3_tier1/v3_usecases enrichments) rebuild **3630/3984 (91%) of `FINAL\_v3.jsonl` exactly**; the ~9% residual is irreducibly lost source data (feature_phrases from overwritten --sample use-case runs, plus five manual CWE-367 overrides), so the committed FINAL_v3.jsonl **remains canonical** as the exact artifact (an equivalent, not byte-identical, bank is rebuildable from the base facts). The published GGUFs plus per-model cards (with a libraries or weakness table, mined-sources, and hand-verified results) are the runnable artifacts; the application-security repos ship both editions and their model.yaml delivery files. A companion evidence ledger maps each claim to the exact file that holds its trail.

16. Open problems and future work

- Run a faithful Docker or Linux GitChameleon re-run to convert the provisional pass@1 into leaderboard-comparable numbers, including the 79 currently-unbuildable problems.
- Port the full retrieval recipe (soft doors, pointers, MMR, HyDE) into the baked Jinja path and verify it in-engine (close F-050); for appsec specifically, find a way to approximate the draft key inside the template, since it is the single biggest served-vs-shipped lever.
- Run the over-deference safety experiment: authority framing over a mix of applicable and inapplicable facts, now more urgent given the stronger thinking-on frame.
- Solve precision-at-density: recover top-k recall as banks grow past about 10k to 20k facts (F-056), and study recall on the large mixed faceted appsec bank, which is the true scaling wall (not latency, not storage).
- Reduce the ~10% thinking-on blank residual on the hardest-reasoning prompts, or route those prompts to the thinking-off edition automatically.
- Re-score the gate-alias-fixed security experts on GPU to quantify the recovery, and run a base vs bank lift for the 4B thinking model.
- Bake and score the built-but-unbaked departments (AI-ML, Web, DevOps, Databases, Web3).

17. Conclusion

Within a carefully scoped and honestly hedged evaluation, the project shows that a small untrained model's deficit on fast-moving libraries is knowledge rather than reasoning, and that injecting the exact missing landmine fact at inference (in the shipped case, from an inverted index baked into the model's own chat-template) recovers much of the gap with no weight change and no external service. The measured lifts are real and consistent across sizes and domains; the in-artifact delivery is a genuine portability and cost advantage; the application-

security expert extends the thesis from version-correctness to security-correctness and, externally benchmarked, beats a much larger cloud model at 12B and 26B; and the reasoning-paradox finding, with its convergence evidence, its authority-framing cure, and now a shipped two-bug fix that makes a thinking-on edition usable, is a contribution of independent interest. The reservations are equally clear and mostly self-identified: a non-official benchmark harness, a documented-recipe vs shipped-system gap that is now named and quantified, a served-loop ceiling that must never be read as the shipped artifact, value concentrated in scarce curation, grounding that proves real but not current, and reasoning and authority claims that rest on small probes with an unaddressed over-deference risk. Treated as what it is (a low-cost, portable knowledge patch plus a sharp observation about reasoning and injected facts), it is a strong foundation for a paper, provided the absolute benchmark numbers are re-established on the official harness and the strong reasoning claims are scaled up and logged.

The project's one-line summary of the thesis: **the bank supplies knowledge, the model supplies reasoning, and the two have to be told which one wins when they disagree.**